

PROJECT AND TEAM INFORMATION

Project Title

(Try to choose a catchy title. Max 20 words).

TeamForge - Skill-Based Hackathon & Project Team Matchmaker

Student / Team Information

Team Name:	TEAMFORGE
Team #:	DSCPP-III-2026-T238
Team member 1 (Team Lead) (Last Name, name: student ID: email, picture):	Fartiyal, Mukul - 2510370466 mukul.fartiyal77@gmail.com 
Team member 2 (Last Name, name: student ID: email, picture):	Bisht, Anubhav - 2510370178 anubhav6875@gmail.com 
Team member 3 (Last Name, name: student ID: email, picture):	Singh, Siddhant - 2510380233 siddhantsingh1890@gmail.com 
Team member 4 (Last Name, name: student ID: email, picture):	Mehul - 2510370093 mehul.200702@gmail.com 

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

(Describe the problem you want to solve and why it is important. Max 300 words).

- Students usually form hackathon, PBL, or project teams through friends, existing groups, or quick messages in class groups.
- This is convenient, but it can leave a team with several people who have similar skills while another required role is not covered.
- Students already know useful information about themselves, such as the skills they can offer, skills they want to learn, experience level, and availability. Projects can also list the roles and skills they need.
- TeamForge brings these two sides together. A student creates a skill profile, while a project or hackathon defines its requirements. The system then retrieves suitable candidates, calculates a compatibility score, and ranks them.
- The team can also be checked for covered skills and remaining gaps. The final choice is still made by the students, but the comparison becomes more structured and easier to review.

State of the Art / Current solution (1 pt)

(Describe how the problem is solved today (if it is). Max 200 words).

- In most college settings, students use WhatsApp, Discord, class groups, friend circles, or personal contacts to find teammates.
- Some teams compare skills manually or keep simple lists in spreadsheets. This becomes harder when the number of students and required skills increases.
- General collaboration and profile tools can help people connect, but they do not necessarily show which available students fit a particular project's requirements or what gaps a proposed team will have.
- TeamForge focuses on this missing step by comparing skills offered by students with the requirements of a specific project and returning a ranked shortlist with visible coverage and gaps.

Project Goals and Milestones (2 pts)

(Describe the project general goals. Include initial milestones as well any other milestones. Max 300 words).

- Create student profiles with skills offered, skills wanted, experience level, and availability.
- Allow a project or hackathon to define required skills, roles, and team-size limits.
- Retrieve and rank suitable candidates using skill coverage, complementarity, experience, availability, and team balance.
- Check a proposed team for covered skills, missing skills, duplicate membership, and size limits before finalization.
- Save profiles, project requirements, and team history in local files so the information remains available between sessions.
- Use TCS-302 data structures and algorithms and TCS-307 OOP concepts as part of the actual application design.
- Milestones: Start with the class model, Qt shell, profile format, and requirement format; then build indexing, scoring, and ranking; add team formation and gap checking; finally test with small hand-checked datasets and demonstrate selected algorithms in the Algorithm Lab.

Project Approach (3 pts)

(Describe how you plan to articulate and design a solution. Including platforms and technologies that you will use. Max 300 words).

- The application is being developed in C++ with Qt for the desktop interface. VS Code/Qt Creator will be used during development, with Git/GitHub for version control.
- Data will initially be stored in structured local files so the main focus stays on the matching logic, persistence, and course concepts rather than cloud infrastructure.
- A hash map will support skill-based candidate lookup. Sets and vectors will organise skills and requirements, while a priority queue will keep strong candidates or team combinations near the top.
- A graph will represent useful compatibility relationships. BST/AVL structures and search-ing/sorting methods can be used for ordered lookup and comparison in the Algorithm Lab.
- The matching score uses five factors: required-skill coverage 40%, complementarity 25%, experience/level 15%, availability 10%, and team diversity/balance 10%.
- The C++ design will connect the application to OOP through classes, inheritance, abstract strategy classes, virtual functions, polymorphism, templates, operator overloading, exception handling, and file I/O.

Project Approach (continued)

Team Task Distribution

Member	Role	Planned contribution
Mukul Fartiyal	Team / Integration Lead	Requirements, overall architecture, coordination, and keeping the project work aligned.
Anubhav Bisht	UI / Testing / Documentation Lead	Qt interface planning, test planning, and maintaining project documentation.
Siddhant Singh	C++ / OOP Lead	C++ class design and the overall application structure, including the OOP side of the system.
Mehul	DSA Lead	Matching logic, searching, hashing, ranking, and the main data-structure work.

Course Integration

Course	Concepts applied	Where used
Data Structures in C	Searching, hashing, queues/priority queues, trees, linked structures, sorting, and graphs.	Candidate retrieval, ranking, coverage checks, team data, and Algorithm Lab.
OOP with C++	Classes and objects, constructors, inheritance, abstract classes, virtual functions, polymorphism, templates, exceptions, and file handling.	Application structure, strategy selection, data models, validation, and local persistence.

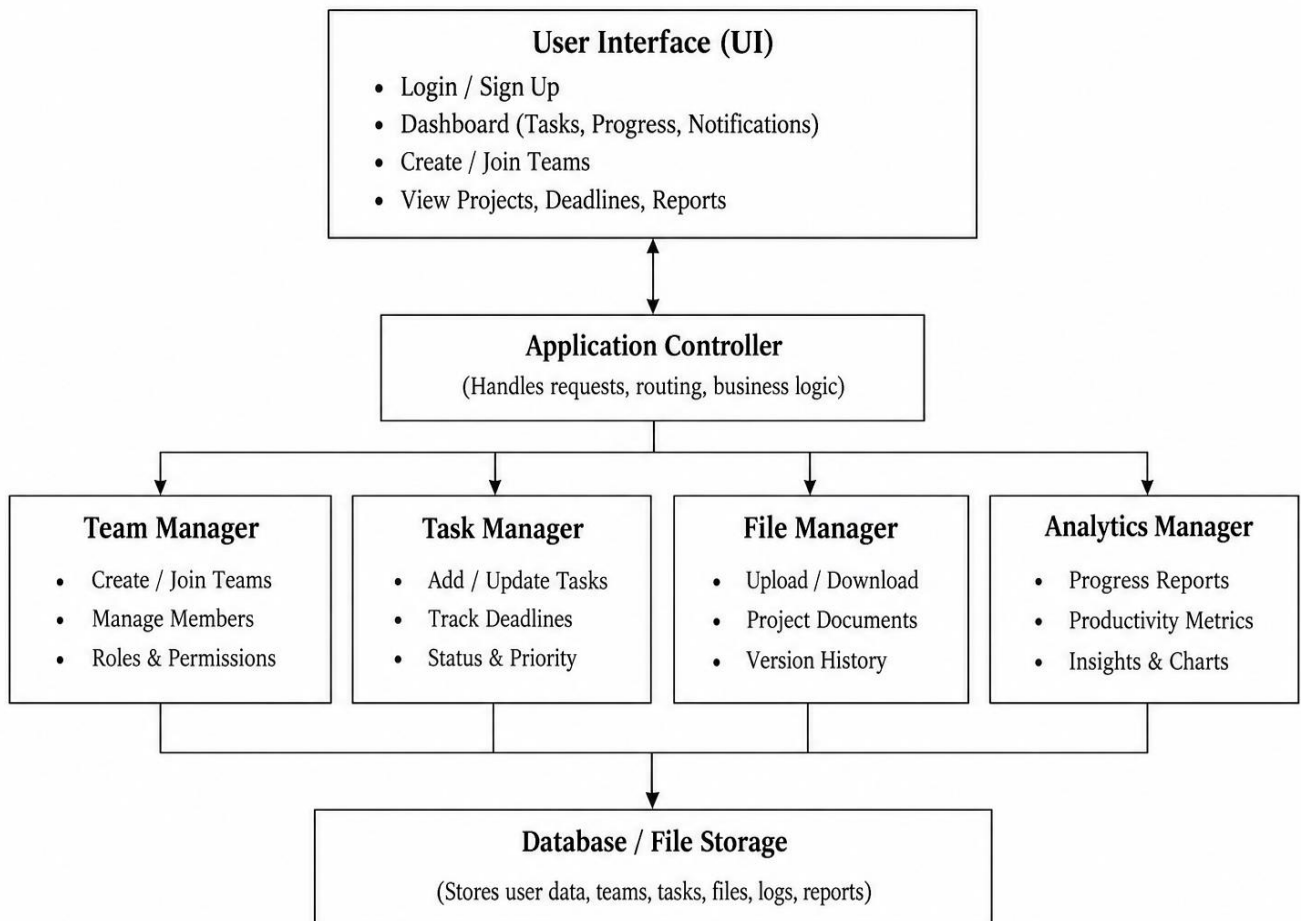
Main design idea: a student profile and a project requirement are the two inputs. The data-structure layer retrieves and organises possible matches. The scoring and priority-queue layer ranks them. The team module checks the selected combination for coverage and missing skills.

Matching Score Weights

Factor	Weight	Use in TeamForge
Required-skill coverage	40%	Measures how many required skills are covered.
Complementarity	25%	Rewards candidates who fill a gap instead of duplicating an existing skill.
Experience / level	15%	Considers relevant experience or proficiency.
Availability	10%	Considers overlap with project time requirements.
Team diversity / balance	10%	Helps avoid excessive concentration of the same role or skill.

System Architecture (High Level Diagram)(2 pts)

(Provide an overview of the system, identifying its main components and interfaces in the form of a diagram using a tool of your choice).



Project Outcome / Deliverables (1 pts)

(Describe what are the outcomes / deliverables of the project. Max 200 words).

- A C++/Qt desktop prototype for creating student skill profiles and project/hackathon re-quirements.
- A matching engine that retrieves relevant candidates, ranks them using the defined score, and reports skill coverage and remaining gaps.
- Team formation and validation features covering team size, duplicate membership, coverage, and skill gaps.
- Source code, project documentation, DSA/OOP mapping, matching-score description, and selected Algorithm Lab work.
- Local persistence for profiles, requirements, and formed-team information, followed by a final demonstration of the planned matching flow.

Assumptions

(Describe the assumptions (if any) you are making to solve the problem. Max 100 words).

- Students provide reasonably accurate information about skills, experience, and availability.
- Project requirements use identifiable skills or roles and include a practical team-size limit.
- The first version runs locally and uses files instead of external APIs or cloud services.
- Candidate data is small enough to be handled in memory, and the score is treated as a decision aid rather than a guarantee of team success.

References

(Provide a list of resources or references you utilised for the completion of this deliverable. You may provide links).

1. Thareja, R. *Data Structures Using C*. Oxford University Press.
2. Balagurusamy, E. *Object Oriented Programming with C++*. McGraw Hill Education.
3. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. *Introduction to Algorithms*. MIT Press.
4. Weiss, M. A. *Data Structures and Algorithm Analysis in C++*. Pearson.
5. Stroustrup, B. *The C++ Programming Language*. 4th ed., Addison-Wesley.
6. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
7. The Qt Company. *Qt Documentation*. <https://doc.qt.io/qt-6/>
8. cppreference.com. *C++ Language and Standard Library Reference*. <https://en.cppreference.com/>

